

Introduzione al C

Lezione 6

Sistemi Operativi - II Modulo (Canale A - L)

Prof. Paolo Zuliani

Dipartimento di Informatica

(Materiale didattico per cortesia del Prof. Emiliano Casalicchio)



SAPIENZA
UNIVERSITÀ DI ROMA



Agenda

- Perché i cicli (iterazione o loop)
- I cicli `while`, `for`, `do-while`
- Blocchi
- Condizionale
- Array



Quando usare i loop

- Ciclo, Loop, Iterazione, Iteration statement
- Quando dobbiamo eseguire ripetutamente un blocco di codice
 - Blocco di codice = corpo del ciclo (*body of loop*)
- Il blocco di codice può contenere istruzioni il cui risultato è dipendente o indipendente dalla specifica iterazione in cui vengono eseguiti

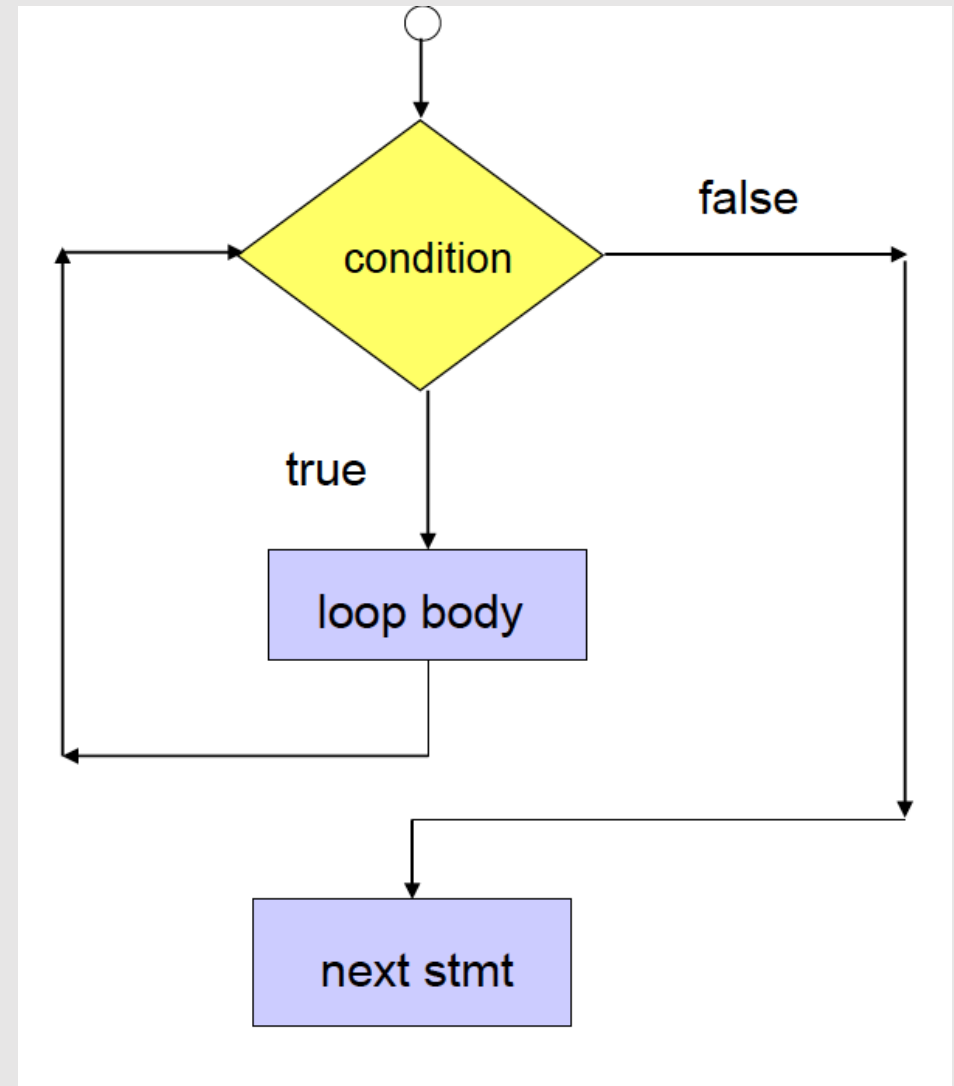
```
loop statement {  
    body of loop  
}
```

Tre tipi di costrutti iterativi

- `while` statement
 - Più flessibile
 - Nessun tipo di restrizione
- `for` statement
 - Ciclo ripetuto un numero finito di volte
- `do-while` statements
 - Esegue il corpo almeno una volta

Ciclo *while*

- *Condition* valutata prima di entrare nel loop
- Se *condition* è vera, allora viene eseguito il corpo una volta e poi viene ri-valutata la condizione
 - Nel corpo deve accadere qualcosa che possa «cambiare» la condizione
 - Un evento esterno deve «cambiare» la condizione
 - Altrimenti ciclo infinito (che potrebbe anche essere una scelta!)
- Il ciclo termina quando *condition* è falsa (oppure quando si usa un *break* – vedi dopo)



Ciclo while

`while (expression)`

`basic block`

- `expression` è la condizione da testare.
Deve risolversi in true o false
- Basic block
 - racchiuso tra `{ }` se contiene più di un istruzione

```
i=0;  
while (i<7)  
    i++;
```

```
i=0;  
while (i<7) {  
    i++;  
    y=y*i;  
}
```

Loop Control Variable (LCV) - variabile di controllo del ciclo

- Ci deve essere almeno una variabile il cui valore controlla la ripetizione del ciclo
 - Altrimenti *condition* è sempre vera (ciclo infinito) o sempre falsa (corpo non eseguito, si passa al comando successivo)
- Per un'esecuzione corretta del `while` le LCV devono
 - Essere dichiarate e inizializzate prima del ciclo
 - Lette nella condizione del ciclo (altrimenti non sono LCV!!!)
 - Aggiornate nel corpo del ciclo in modo che prima o poi la condizione diventi falsa
 - Altrimenti loop infinito (in alcuni casi può essere una scelta voluta)



Ciclo controllato da variabile contatore

- Il corpo viene eseguito un numero definito di volte, noto *a priori*
- Requisiti
 - LCV inizializzata ad un valore iniziale
 - Condizione testa il valore finale di LCV
 - Incremento/decremento costante di LCV

```
int count; // LCV: Loop Control Variable
int num1, num2, sum;

count = 0; // 1. Initialize LCV
while (count < 5) { // 2. Test LCV
    scanf("%d %d", &num1, &num2);
    sum = num1 + num2;
    printf("%d + %d = %d\n", num1, num2, sum);
    count = count + 1; // 3. Increment LCV
}
```

EXECUTION CHART		
count	count<5	repetition
0	true	1
1	true	2
2	true	3
3	true	4
4	true	5
5	false	

Errori comuni

- Analizzate questo codice (magari copiatelo ed eseguitelo)
- Cosa c'è di sbagliato? Perché il programma rimane bloccato dopo che avete inserito un numero?

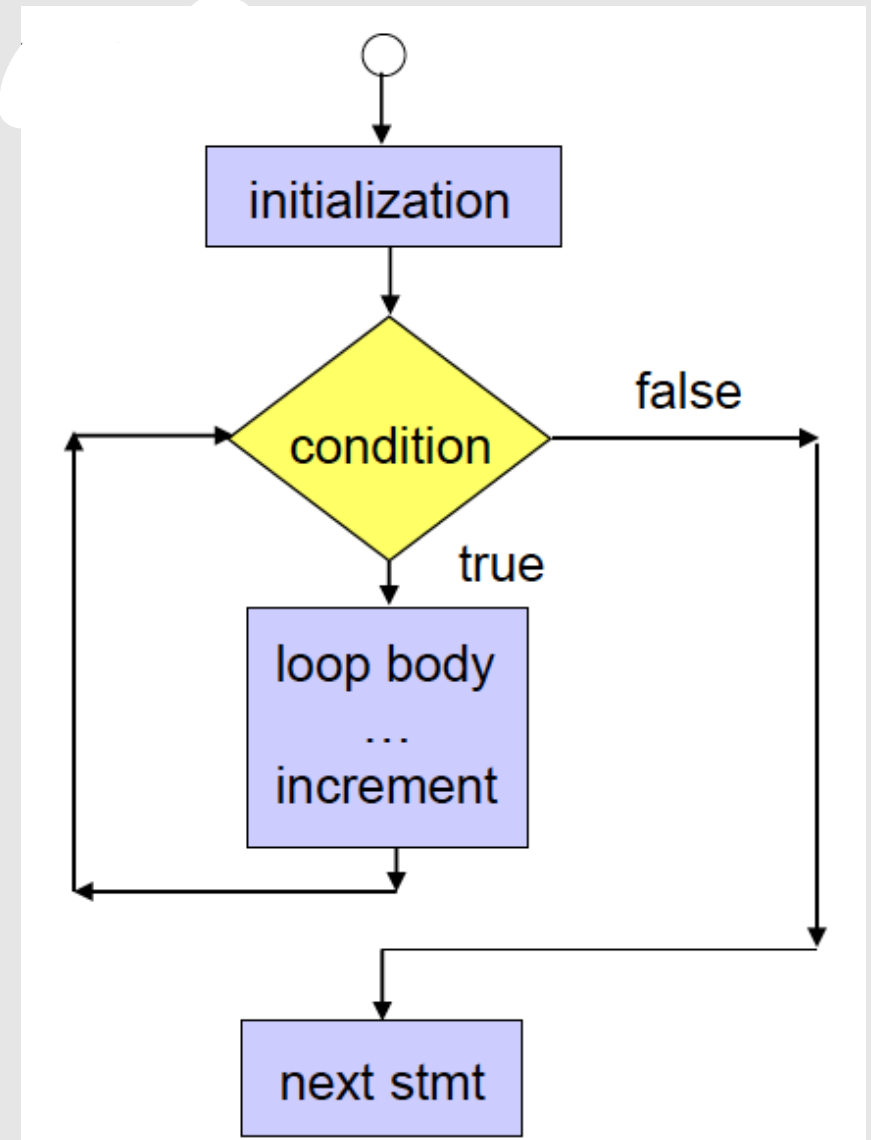
```
// Echo numbers entered back to user
printf("Enter number or zero to end: ");
scanf("%d", &num);
while (num != 0);
{
    printf("Number is %d\n\n", num);
    printf("Enter another number or zero to end: ");
    scanf("%d", &num);
}
```

Ciclo `for`

- Noto *a priori* il numero di iterazioni
 - tale comportamento può essere sovvertito
- Inizializzazione LCV, test condizione su LCV e incremento/decremento LCV sono definite nello statement

```
for (initialization; test; increment)  
    basic block
```

Ad es: `for (i=1; i<5; i++)`



Ciclo for: esempi

- for

```
int counter;  
for (counter = 1; counter <= 10; counter++)  
{  
    printf("%d\n", counter);  
}
```

- while

```
int counter;  
counter = 1;  
while (counter <= 10)  
{  
    printf("%d\n", counter);  
    counter++;  
}
```

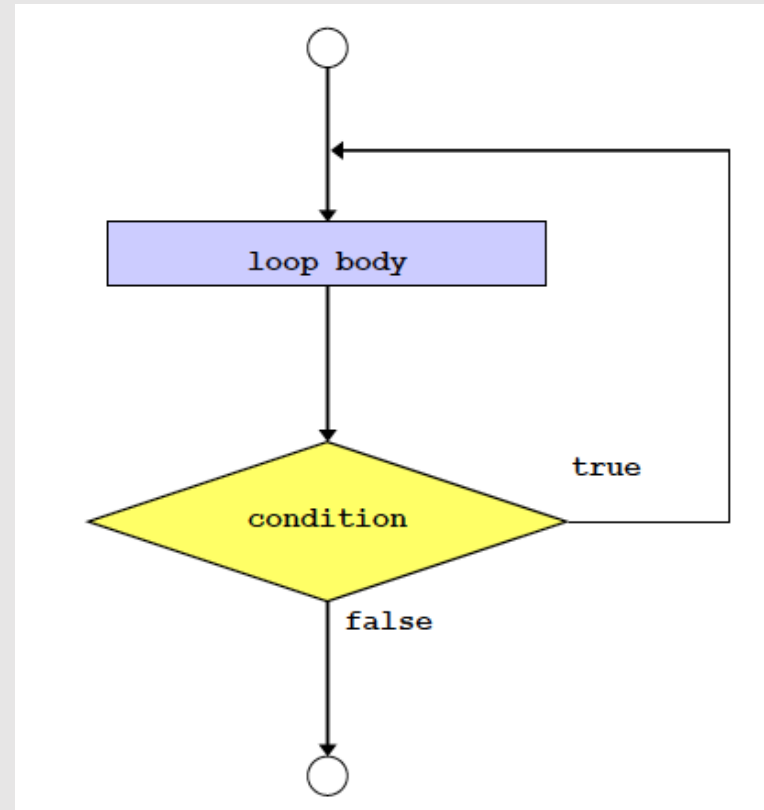
Ciclo `for`: esempi

- **Esercizio 2:** Quante volte viene eseguito il ciclo seguente?

```
int count;  
for (count = 0; count < 3; count++) {  
    printf("Gnam %d - ", count+1);  
    printf("Yum!\n");  
}
```

Ciclo do-while

- Simile al `while`, ma la condizione è testata alla fine
 - Il corpo è eseguito almeno una volta
- ```
do {
 statements
} while (condition);
```



# Ciclo do-while (cont.)

- Esempio 1 - conta interi da 1 a 10
- Esempio 2 - controlla che l'input sia corretto e non termina fino a che non si inserisce un valore corretto

```
int counter = 1;
do {
 printf("%d\n", counter);
 counter ++;
} while (counter <= 10);
```

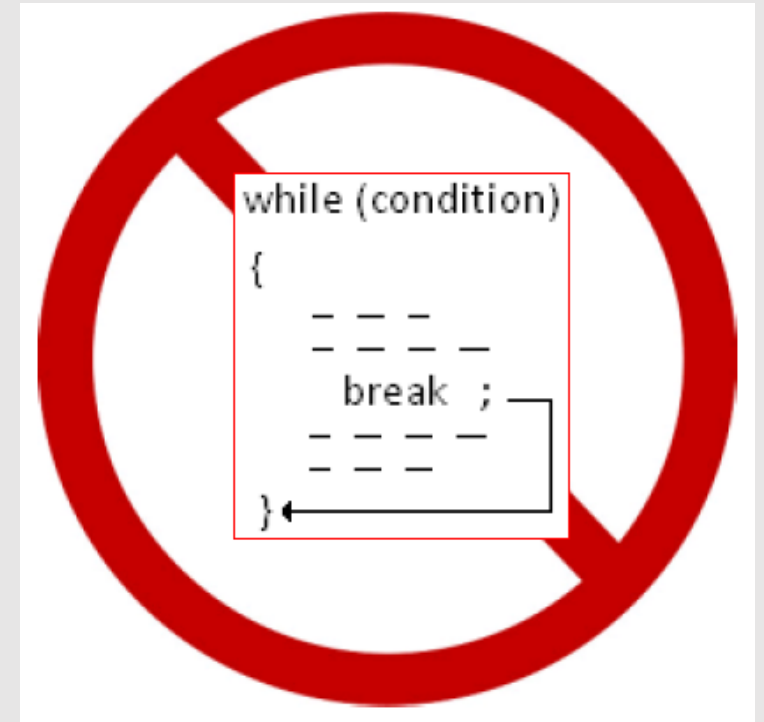
```
do {
 printf("Enter a positive weight: ");
 scanf("%d", &weight);
} while (weight <= 0);
```

- **Esercizio 3.** Provare ad implementare l'esempio 2 (slide precedente) con il `while` loop
- **Esercizio 4.** Scrivere un programma che riceva come input (del comando) un valore  $n$  che indica il numero di dati che debbono essere letti da `stdin`. Per ogni dato letto da `stdin` verifica e impone che esso sia maggiore di 0 e minore di 10. Dopo aver letto gli  $n$  dati di input ne calcola e stampa il valore della somma ed il valore medio. Implementare la soluzione con 3 diverse combinazioni di cicli:
  - Solo `while`
  - `While` + `do-while`
  - `For` + `while` o `do-while`

NOTA: realizzare i programmi con il minor numero di righe di codice.

# break

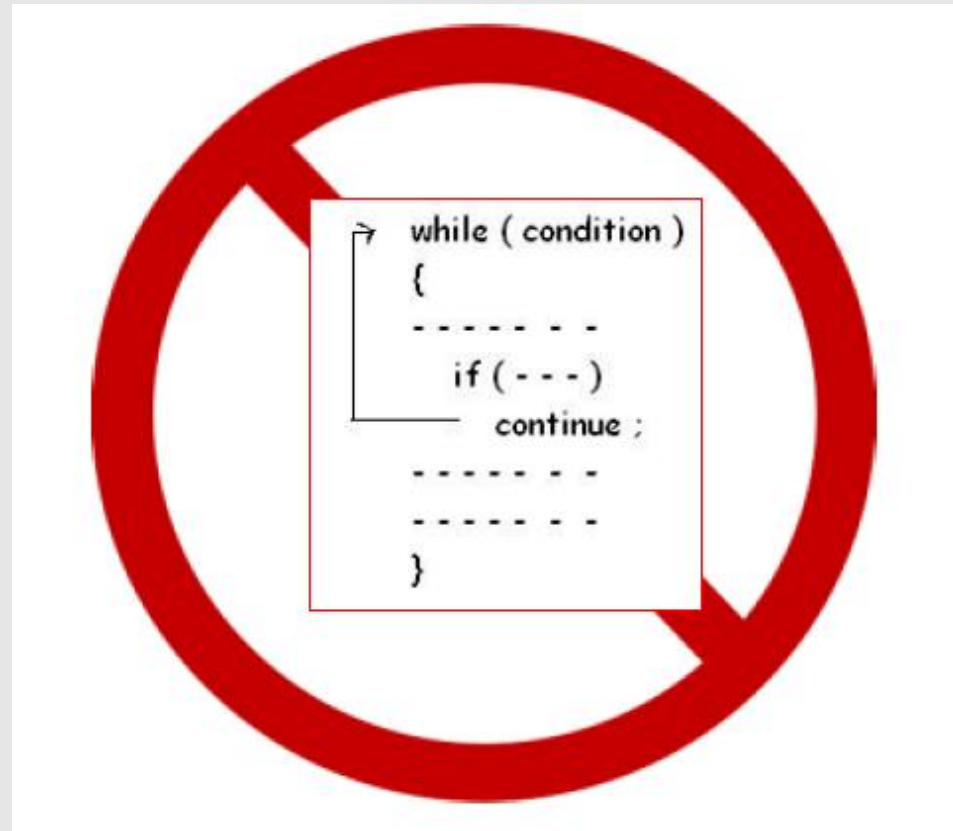
- Si usa per uscire da un ciclo
  - `for`, `while`, `do-while`
- Si usa per uscire da uno `switch` (lo vedremo più in là)
- Quando viene eseguito `break` il ciclo termina *indipendentemente* dal valore della condizione





# continue

- Il controllo passa all'iterazione successiva



# Blocchi



# Blocchi

- **blocco** = porzione di codice racchiusa da { }
- Le variabili definite in un blocco sono **locali** al blocco:
  - Vengono allocate e sono accessibili solo quando il blocco è eseguito
  - Non sono più accessibili dopo che il blocco è stato eseguito
  - Nel blocco, «oscurano» eventuali variabili con lo stesso nome definite al di fuori del blocco

# Blocchi

```
int main() {
 {
 int i = 0;
 printf("i = %d\n", ++i);
 }
 return 0;
}
```

Alloca la variabile `i`

Cosa stampa?  
`i = 1`

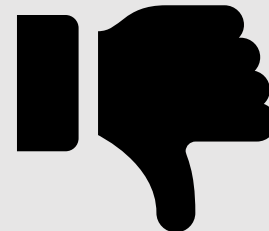
`i` viene deallocata

# Blocchi

```
int main() {
 {
 int i = 0;
 printf("i = %d\n", ++i);
 }
 printf("i = %d\n", ++i);
 return 0;
}
```

Questo programma è **errato sintatticamente** e il compilatore ve lo dice: provate!

In questo punto non c'è alcuna variabile `i` 'visibile' al codice.



# Blocchi: «Oscuramento» di variabili omonime

```
int main() {
 int i = -1;
 {
 int i = 0;
 printf("i = %d\n", i++);
 }
 printf("i = %d\n", i);
 return 0;
}
```

Cosa stampa?

i = 0

Cosa stampa?

i = -1

# Blocchi e ciclo for

```
int i = 0;
```

```
...
```

```
for (i = 1; i < 100; i++)
```

```
{
```

```
...
```

```
}
```



In entrambi i casi c'è una variabile `i`  
'visibile' al codice.

```
int i = 0;
```

```
...
```

```
{
```

```
...
```

```
for (i = 1; i < 100; i++)
```

```
{
```

```
...
```

```
}
```

```
}
```



# Blocchi e ciclo `for`

```
int i = 0;
```

```
...
```

```
for (int i = 1; i < 100; i++)
```

```
{
```

```
...
```

```
}
```

Alloca la variabile `i` in nuovo blocco!



# Blocchi e ciclo for

```
int main() {
 int i = -1;
 for (int i = 1; i < 3; i++)
 printf("i = %d\n", i);
 printf("i = %d\n", i);
 return 0;
}
```

Cosa stampa?

i = 1  
i = 2

Cosa stampa?

i = -1

# Blocchi e ciclo `for`

## Esercizio:

Trasformate, preservandone la semantica, il codice seguente inserendo la dichiarazione `int i = 1` del ciclo `for` in un nuovo blocco

```
int main() {
 int i = -1;
 for (int i = 1; i < 3; i++)
 printf("i = %d\n", i);
 printf("i = %d\n", i);
 return 0;
}
```

# Blocchi e ciclo for

## Soluzione:

```
int main() {
 int i = -1;
 {
 int i;
 for (i = 1; i < 3; i++)
 printf("i = %d\n", i);
 }
 printf("i = %d\n", i);
 return 0;
}
```

# Condizionale



# Condizionale

```
if (espressione_booleana) {
 // ramo espressione_booleana VERA
} else {
 // ramo espressione_booleana FALSA
 // il ramo else è opzionale
}
```

# Condizionale (annidato)

```
if (espressione_booleana_1) {
 // un codice
} else if (espressione_booleana_2) {
 // un altro codice
} else if (espressione_booleana_3) {
 // ancora codice
} else { // bla bla
}
```

# Array



# Array

- Un tipo di dato ‘strutturato’
- Letteralmente ‘schiera’ di variabili
  - Dello stesso tipo
  - Occupano una regione contigua di memoria
  - La dimensione rimane inalterata nel corso dell’esecuzione del codice
- Usabili singolarmente o come una singola entità
- Essenzialmente una lista finita indicizzata da un numero intero  $\geq 0$



# Array: Dichiarazione

- La memoria per gli array viene interamente allocata quando il programma è caricato in memoria (sono variabili *statiche*)

```
unsigned int data[3]; // una data gg/mm/aaaa
char alfabeto[21]; // l'alfabeto italiano
double temp_max[365]; // temperatura max di
 // ogni giorno dell'anno
```

# Array: Dichiarazione

- Usate costanti o il preprocessore per dichiarazioni più chiare e maneggevoli

```
const unsigned int GIORNI=365;
double temp_max[GIORNI];
```

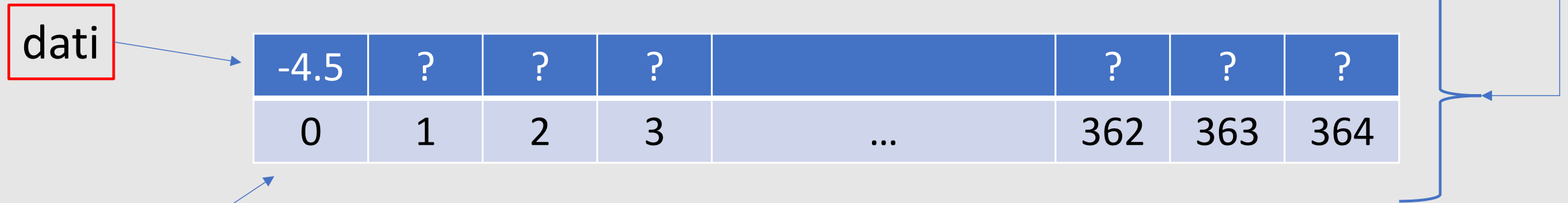
oppure, (quasi) equivalentemente:

```
#define GIORNI 365
double temp_max[GIORNI];
```

Direttiva del preprocessore: è una manipolazione puramente *sintattica* del codice

# Array: Dichiarazione

```
double temp_max[365];
temp_max[0] = -4.5;
```



indici

Questa è un'astrazione utile al programmatore: in C, la variabile `temp_max` è un puntatore al primo elemento dell'array!

# Array: Dichiarazione

- Autoinizializzazione di array:

```
double tmax_week[7]={-2.3, 0.97};
```

|      |       |     |     |     |     |     |
|------|-------|-----|-----|-----|-----|-----|
| -2.3 | -0.97 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| 0    | 1     | 2   | 3   | 4   | 5   | 6   |



```
// inizializza tutto a 0
double tmax_week[7]={};
```

# Array: Dichiarazione

- Array multi-dimensionali:

```
#define MESI 12
#define GIORNI_M 31
double temp_max[MESI][GIORNI_M]; //372 elementi
```

- L'accesso è via due indici: 'righe' e 'colonne', come per una matrice
- Si possono definire array a tre dimensioni e oltre

# Array: Uso

```
double temp_max[365];
```

```
int i = 0;
```

```
temp_max[i] = -4.5;
```

```
temp_max[++i] = -3.8;
```

```
printf("temp_max[%d] = %lf\n", i, temp_max[i]);
```

Un'espressione che  
ritorna un intero

Cosa stampa?

temp\_max[1] = -3.8

|      |      |   |   |     |     |     |     |
|------|------|---|---|-----|-----|-----|-----|
| -4.5 | -3.8 | ? | ? |     | ?   | ?   | ?   |
| 0    | 1    | 2 | 3 | ... | 362 | 363 | 364 |

# Array: Uso della memoria

```
double temp_max[365];
temp_max[0] = -4.5;
temp_max[1] = -3.8;
printf("%ld bytes\n", sizeof(temp_max));
```

Cosa stampa?  
2920 bytes



|      |      |   |   |     |     |     |     |
|------|------|---|---|-----|-----|-----|-----|
| -4.5 | -3.8 | ? | ? |     | ?   | ?   | ?   |
| 0    | 1    | 2 | 3 | ... | 362 | 363 | 364 |

La memoria per l'**intero** array è allocata all'inizio del programma e `sizeof` ritorna la dimensione di tale memoria.

Il programmatore (voi!) deve tener traccia di quali elementi dell'array sono stati scritti.

# Array: Uso

- Gli array sono pensati per l'uso con il ciclo `for`
- Esempio: inizializzazione a zero di un intero array

```
#define GIORNI 365
double temp_max[GIORNI];
```

```
for (int i = 0; i < GIORNI; i++)
 temp_max[i] = 0.0;
```

**ATTENTI** a non sovrascrivere memoria che non vi è stata allocata!! Il compilatore C non controlla per tali errori.



# Array: Uso improprio

- Cosa succede se leggo/scrivo oltre l'ultimo elemento di un array?
  1. Il programma potrebbe continuare, con/senza errori
  2. Il sistema operativo intercetta la violazione e uccide il processo
- Provate!

```
#define GIORNI 365
double temp_max[GIORNI];
for (int i = 0; i < GIORNI; i++)
 temp_max[i] = 0.0;
```

1. Aumentate questo valore finché il processo non viene ucciso.
2. Provate a stampare elementi di `temp_max` invece che scriverli.

# Array: Esercizi

- Implementare una ricerca sequenziale in un array di interi definito all'interno di un programma:
  1. Usate un ciclo `while`
  2. Usate un ciclo `for`
- Scrivete un programma che calcoli e stampi la media aritmetica di un array di max 10 numeri in virgola mobile:
  1. Definite l'array all'interno del programma
  2. Leggete con `scanf` l'array da stdin, terminato da CTRL-d (ricordatevi che potete ridirezionare lo stdin)

# Domande?

